Ex. No: 12

Title: CNN Architecture and Hyperparameter Comparison

Packages: TensorFlow, Keras, NumPy, Matplotlib

AIM:

To compare different CNN architectures and hyperparameters (number of filters, number of convolution layers and learning rate).

ALGORITHM:

1. Start the program and import the required libraries.
2. Load the digits images, reshape and split them into training and testing sets.
3. Create a function to build a CNN with the given number of filters, convolution layers and learning rate.
4. Define four different configurations.
5. Train each CNN for 15 epochs.
6. Evaluate each CNN and print its test accuracy.
7. Plot the accuracy of all configurations as a bar chart.
8. Display the output and stop.

PROGRAM:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers

X, y = load_digits(return_X_y=True)
X = X.reshape(-1, 8, 8, 1) / 16.0
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

def build_cnn(filters, conv_layers, lr):
    model = keras.Sequential([layers.Input(shape=(8, 8, 1))])
    for _ in range(conv_layers):
        model.add(layers.Conv2D(filters, (3, 3), padding="same", activation="relu"))
        model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Flatten())
    model.add(layers.Dense(32, activation="relu"))
    model.add(layers.Dense(10, activation="softmax"))
    model.compile(optimizer=keras.optimizers.Adam(lr),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model

# (filters, conv layers, learning rate)
configs = {
    "1 Conv, 8 filters, lr=0.001": (8, 1, 0.001),
    "1 Conv, 32 filters, lr=0.001": (32, 1, 0.001),
    "2 Conv, 16 filters, lr=0.001": (16, 2, 0.001),
    "2 Conv, 16 filters, lr=0.01": (16, 2, 0.01),
}

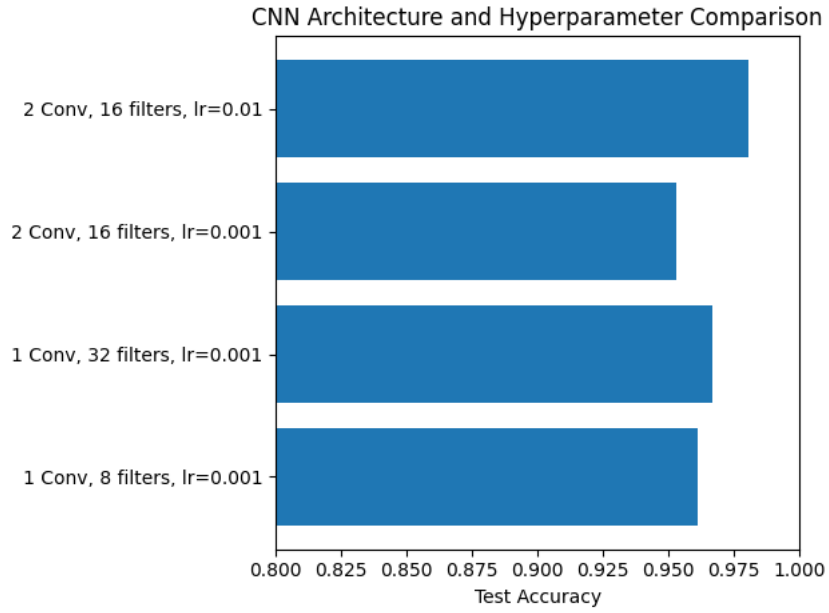
names, accs = [], []
```

```
for name, (f, c, lr) in configs.items():
    model = build_cnn(f, c, lr)
    model.fit(X_train, y_train, epochs=15, verbose=0)
    acc = model.evaluate(X_test, y_test, verbose=0)[1]
    print(name, "- Test Accuracy =", round(acc, 4))
    names.append(name)
    accs.append(acc)

plt.barh(names, accs)
plt.xlim(0.8, 1.0)
plt.xlabel("Test Accuracy")
plt.title("CNN Architecture and Hyperparameter Comparison")
plt.tight_layout()
plt.show()
```

OUTPUT:

```
1 Conv, 8 filters, lr=0.001 - Test Accuracy = 0.9611
1 Conv, 32 filters, lr=0.001 - Test Accuracy = 0.9667
2 Conv, 16 filters, lr=0.001 - Test Accuracy = 0.9528
2 Conv, 16 filters, lr=0.01 - Test Accuracy = 0.9806
```



RESULT:

Different CNN architectures and hyperparameters were compared successfully, and the configuration with the highest test accuracy was identified.